

1. Escolha da LLM

- Ferramenta: ChatGPT
- Data: 10/06/2026
- Versão: Gratuita
- Acesso à internet: Não utilizado durante a realização da atividade.

2. Primeira tarefa: explicação conceitual

2.1. Prompt simples: Explique o que é recursão em Computação.

Resposta:

Recursão é uma técnica de programação em que uma função resolve um problema chamando a si mesma para resolver versões menores do mesmo problema.

A ideia central é dividir um problema grande em subproblemas semelhantes até chegar a uma situação simples que pode ser resolvida diretamente.

Componentes de uma função recursiva

Uma função recursiva normalmente possui:

1. **Caso base:** condição que encerra as chamadas recursivas.
2. **Passo recursivo:** parte que chama a própria função com uma versão menor ou mais simples do problema.

Exemplo: fatorial

O fatorial de um número n é definido como:

- $0! = 10! = 10! = 1$
- $n! = n \times (n-1)! \quad n! = n \times (n-1)! = n \times (n-1)!$

Isso é naturalmente recursivo.

$$n! = n(n-1)! \quad n! = n \times (n-1)! = n(n-1)!$$

Exemplo em Python:

```
def fatorial(n):  
    if n == 0:          # caso base  
        return 1  
    return n * fatorial(n - 1)  # passo recursivo  
  
print(fatorial(5))    # 120
```

Execução simplificada:

```
fatorial(5)
= 5 * fatorial(4)
= 5 * 4 * fatorial(3)
= 5 * 4 * 3 * fatorial(2)
= 5 * 4 * 3 * 2 * fatorial(1)
= 5 * 4 * 3 * 2 * 1 * fatorial(0)
= 5 * 4 * 3 * 2 * 1 * 1
= 120
```

Outro exemplo: sequência de Fibonacci

Cada termo é a soma dos dois anteriores:

$$F(n)=F(n-1)+F(n-2) \quad F(n) = F(n-1) + F(n-2)$$

com:

- $F(0)=0$
- $F(1)=1$

Código:

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Como o computador executa a recursão

Cada chamada da função é armazenada em uma estrutura chamada **pilha de chamadas** (*call stack*). Quando uma função chama a si mesma:

1. O estado atual é guardado na pilha.
2. A nova chamada é executada.
3. Quando o caso base é alcançado, as chamadas retornam em ordem inversa.

Vantagens

- Código mais próximo da definição matemática de alguns problemas.
- Muito útil para estruturas hierárquicas como árvores e diretórios.
- Pode tornar algoritmos complexos mais simples de entender.

Desvantagens

- Consome memória na pilha de chamadas.
- Pode ser mais lenta que uma solução iterativa.
- Se não houver um caso base correto, ocorre uma recursão infinita, levando a um erro de estouro de pilha (*stack overflow*).

Quando usar

Recursão é especialmente útil em problemas como:

- Percorrer árvores.
- Busca em grafos.
- Algoritmos de divisão e conquista, como Merge Sort e Quick Sort.
- Processamento de estruturas aninhadas (pastas, expressões matemáticas, JSON etc.).

Uma forma intuitiva de pensar em recursão é: "**para resolver o problema atual, resolva primeiro uma versão menor do mesmo problema.**" O desafio é garantir que essas versões menores eventualmente alcancem o caso base.

- Ferramenta: ChatGPT
- Data: 10/06/2026

2.2. Prompt mais elaborado: Explique o conceito de recursão em Computação para um estudante do primeiro semestre de Ciência da Computação. Utilize linguagem simples, apresente uma definição, um exemplo em C e liste vantagens e desvantagens.

Resposta:

Recursão em Computação

Definição

Recursão é uma técnica de programação em que uma função chama a si mesma para resolver um problema. A ideia é dividir um problema grande em versões menores do mesmo problema, até chegar a uma situação simples que pode ser resolvida diretamente.

Para que uma função recursiva funcione corretamente, ela precisa de:

1. **Caso base:** condição que encerra as chamadas recursivas.
2. **Passo recursivo:** parte em que a função chama a si mesma com um problema menor.

Sem um caso base, a função continuaria chamando a si mesma indefinidamente, causando erro de execução.

Exemplo em C

Um exemplo clássico é o cálculo do fatorial de um número.

Matematicamente:

- $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
- $1! = 1$ (caso base)

Código

```
#include <stdio.h>

int fatorial(int n) {
    if (n <= 1) {
        return 1; // caso base
    }

    return n * fatorial(n - 1); // chamada recursiva
}

int main() {
    int numero = 5;

    printf("Fatorial de %d = %d\n", numero, fatorial(numero));

    return 0;
}
```

Como a execução acontece para `fatorial(5)`

```
fatorial(5)
= 5 * fatorial(4)
= 5 * (4 * fatorial(3))
= 5 * (4 * (3 * fatorial(2)))
= 5 * (4 * (3 * (2 * fatorial(1))))
= 5 * (4 * (3 * (2 * 1)))
= 5 * (4 * (3 * 2))
= 120
```

A função continua chamando a si mesma até chegar ao caso base (`fatorial(1)`).

Vantagens da Recursão

- **Código mais simples e elegante** para certos problemas.
- Facilita a implementação de algoritmos que possuem estrutura naturalmente recursiva.
- Muito útil para trabalhar com:
 - árvores;
 - grafos;
 - divisão e conquista (por exemplo, Merge Sort e Quick Sort).
- Pode tornar a lógica mais fácil de entender do que soluções iterativas complexas.

Desvantagens da Recursão

- Consome mais memória devido ao armazenamento das chamadas na pilha de execução (*call stack*).
- Pode ser mais lenta do que uma solução iterativa equivalente.
- Se o caso base estiver incorreto, pode ocorrer **recursão infinita**.
- Muitas chamadas recursivas podem causar **estouro de pilha** (*stack overflow*).

Resumo

A recursão é uma técnica em que uma função resolve um problema chamando a si mesma para resolver versões menores desse mesmo problema. Ela exige um **caso base** para parar as chamadas e um **passo recursivo** para avançar na solução. Embora seja uma ferramenta poderosa e elegante para vários algoritmos, deve ser usada com cuidado devido ao consumo adicional de memória e ao risco de estouro de pilha.

- **Ferramenta: ChatGPT**
- **Data: 10/06/2026**

2.3. Comparação entre as respostas:

As duas respostas explicam corretamente o conceito de recursão e apresentam os elementos fundamentais de uma função recursiva, como o caso base e o passo recursivo.

A principal diferença é que a primeira resposta é mais detalhada, incluindo exemplos em Python, Fibonacci e uma explicação sobre a pilha de chamadas. Já a segunda resposta é mais objetiva e focada no público-alvo especificado no prompt, utilizando linguagem mais simples e um exemplo em C.

Além disso, a segunda resposta possui uma estrutura mais organizada, com seções bem definidas para definição, exemplo, vantagens e desvantagens. Isso facilita a leitura e a compreensão do conteúdo.

A comparação mostra que um prompt mais elaborado produz respostas mais adequadas às necessidades do usuário, evidenciando a importância de fornecer contexto e instruções claras a LLM.

3. Geração de um pequeno programa

4.

Prompt utilizado: Crie um programa em C que leia um número inteiro e informe se ele é par ou ímpar.

Código gerado: Aqui está um programa simples em C que lê um número inteiro e informa se ele é par ou ímpar:

```
#include <stdio.h>

int main() {
    int numero;

    printf("Digite um número inteiro: ");
    scanf("%d", &numero);

    if (numero % 2 == 0) {
        printf("O número %d é par.\n", numero);
    } else {
        printf("O número %d é ímpar.\n", numero);
    }

    return 0;
}
```

- **Ferramenta:** ChatGPT
- **Data:** 10/06/2026

Explicação do código: O código recebe o input de um número inteiro por meio da função `scanf()`, guarda esse número na variável “numero” e verifica se o resto da divisão do número por 2, utilizando o operador `%`, resulta em 0. Se resultar em 0, o programa informa que o número é par, caso contrário, informa que é ímpar.

Dois testes realizados:

1.

Digite um número inteiro: 7
O número 7 é ímpar.

2.

Digite um número inteiro: 10
O número 10 é par.

Análise crítica da solução: A solução produzida atende corretamente ao objetivo proposto, identificando se um número inteiro é par ou ímpar de forma simples e eficiente. O código é fácil de compreender e utiliza conceitos básicos da linguagem C, sendo adequado para iniciantes. Entretanto, o programa não realiza a validação da entrada do usuário, assumindo que será

informado um número inteiro válido. Apesar dessa limitação a solução funciona corretamente para os testes realizados.

4. Melhorando o prompt

Prompt melhorado:

Crie um programa em C que leia um número inteiro e informe se ele é par ou ímpar.

O código deve conter comentários explicando as partes principais, apresentar uma explicação adequada para um estudante iniciante em programação, incluir exemplos de entrada e saída e realizar tratamento de erros para entradas inválidas.

Nova resposta:

A seguir está um programa em C que lê um número inteiro, verifica se ele é par ou ímpar e realiza um tratamento básico de erros para entradas inválidas.

```
#include <stdio.h>

int main() {
    int numero;

    // Solicita ao usuário que digite um número inteiro
    printf("Digite um número inteiro: ");

    // Verifica se a leitura foi bem-sucedida
    if (scanf("%d", &numero) != 1) {
        printf("Erro: entrada inválida. Digite apenas números inteiros.\n");
        return 1; // Encerra o programa indicando erro
    }

    // Verifica se o número é par ou ímpar
    if (numero % 2 == 0) {
        printf("O número %d é par.\n", numero);
    } else {
        printf("O número %d é ímpar.\n", numero);
    }

    return 0; // Encerra o programa com sucesso
}
```

[Explicação para iniciantes](#)

1. Inclusão da biblioteca

```
#include <stdio.h>
```

A biblioteca `stdio.h` fornece funções de entrada e saída, como `printf()` e `scanf()`.

2. Função principal

```
int main() {
```

Todo programa em C começa a execução pela função `main()`.

3. Declaração da variável

```
int numero;
```

Cria uma variável chamada `numero` do tipo inteiro (`int`), onde será armazenado o valor digitado pelo usuário.

4. Entrada de dados

```
printf("Digite um número inteiro: ");
```

Exibe uma mensagem na tela solicitando a entrada do usuário.

```
if (scanf("%d", &numero) != 1)
```

A função `scanf()` tenta ler um número inteiro.

- `%d` indica que esperamos um inteiro.
- `&numero` fornece o endereço da variável onde o valor será armazenado.
- Se o usuário digitar algo que não seja um número inteiro, `scanf()` não conseguirá realizar a leitura corretamente.

5. Tratamento de erro

```
printf("Erro: entrada inválida. Digite apenas números inteiros.\n");  
return 1;
```

Caso a entrada seja inválida, o programa exibe uma mensagem de erro e termina sua execução.

6. Verificação de paridade

```
if (numero % 2 == 0)
```

O operador `%` calcula o resto da divisão.

- Se `numero % 2` for igual a 0, o número é par.
- Caso contrário, o número é ímpar.

7. Encerramento do programa

```
return 0;
```

Indica que o programa terminou corretamente.

Exemplos de execução

Exemplo 1: Número par

Entrada

```
Digite um número inteiro: 8
```

Saída

```
O número 8 é par.
```

Exemplo 2: Número ímpar

Entrada

```
Digite um número inteiro: 15
```

Saída

```
O número 15 é ímpar.
```

Exemplo 3: Entrada inválida

Entrada

```
Digite um número inteiro: abc
```

Saída

```
Erro: entrada inválida. Digite apenas números inteiros.
```

Conceito importante

A lógica utilizada é baseada na propriedade matemática de que:

- Números pares são divisíveis por 2 sem deixar resto.
- Números ímpares deixam resto 1 quando divididos por 2.

Por isso, a expressão:

```
numero % 2 == 0
```

é uma das formas mais comuns e eficientes de verificar se um número é par em C.

- Ferramenta: ChatGPT
- Data: 10/06/2026

Comparação das respostas:

A nova resposta foi mais completa e útil do que a resposta anterior. Enquanto a primeira versão apenas apresentava o código e uma breve explicação do funcionamento, a segunda incluiu comentários no código, uma explicação detalhada de cada parte do programa, exemplos de entrada e saída e tratamento de erros para entradas inválidas.

Além disso, a nova resposta foi elaborada para um estudante iniciante, utilizando uma linguagem mais didática e explicando conceitos básicos da linguagem C, como o uso de `scanf()`, `printf()` e do operador `%`. O tratamento de erros também tornou o programa mais robusto, pois evita comportamentos inesperados quando o usuário digita uma entrada inválida.

Essa comparação demonstra que a adição de mais contexto e requisitos ao prompt resultou em uma resposta mais completa, organizada e adequada para fins de aprendizado.

5. Verificação crítica

A resposta analisada foi a primeira solução gerada para o prompt: "Crie um programa em C que leia um número inteiro e informe se ele é par ou ímpar."

5.1 A resposta estava correta?

Sim. O programa gerado identificava corretamente se um número inteiro era par ou ímpar utilizando o operador módulo (%).

5.2 Havia informações incompletas ou incorretas?

Não havia erros na lógica do programa. Entretanto, a primeira versão não realizava validação da entrada do usuário, assumindo que seria fornecido um número inteiro válido.

5.3 Como a resposta foi verificada?

A resposta foi verificada por meio da compilação e execução do código, realizando testes com números pares e ímpares e comparando os resultados obtidos com os resultados esperados.

5.4 A LLM demonstrou excesso de confiança?

Não. A resposta apresentou uma solução simples e adequada ao problema proposto, sem fazer afirmações exageradas sobre sua qualidade ou abrangência.

5.5 Como o prompt poderia ser melhorado?

O prompt poderia especificar requisitos adicionais, como tratamento de erros, comentários explicativos, exemplos de entrada e saída e um nível de explicação adequado para iniciantes. Isso tornaria a resposta mais completa e útil para fins de aprendizado.

Parte 6 – Uso responsável de LLMs

6.1 Por que não devemos copiar respostas sem revisão?

Não devemos copiar respostas sem revisão porque as LLMs podem cometer erros ou fornecer informações incompletas. Mesmo quando a resposta parece correta, é importante verificar o conteúdo para garantir que ele esteja adequado ao contexto e aos objetivos do trabalho.

6.2 Por que é importante informar os prompts utilizados?

Informar os prompts utilizados traz transparência ao trabalho e permite que outras pessoas entendam como as respostas foram obtidas. Além disso, ajuda a demonstrar que a ferramenta foi utilizada de forma consciente e responsável.

6.3 Em quais situações as LLMs ajudam no aprendizado?

As LLMs ajudam principalmente na compreensão de conceitos, na geração de exemplos, na explicação de códigos e na resolução de dúvidas. Elas também podem auxiliar na organização de ideias e servir como apoio durante os estudos.

6.4 Em quais situações elas podem atrapalhar?

Elas podem atrapalhar quando o usuário aceita as respostas sem questionar ou verificar as informações apresentadas. Também podem prejudicar o aprendizado se forem utilizadas para substituir completamente o estudo e a prática dos conteúdos.

6.5 O que significa utilizar uma LLM como apoio e não como substituta do raciocínio?

Significa utilizar a ferramenta para auxiliar na compreensão de conceitos, esclarecer dúvidas e fornecer exemplos, mas sem deixar de analisar criticamente as respostas. O aprendizado e a tomada de decisões continuam sendo responsabilidade do usuário, que deve compreender e validar as informações recebidas.

7. Reflexão sobre o uso de IA Generativa

7.1 O que você aprendeu sobre interação com LLMs?

Aprendi que a forma como o prompt é escrito influencia diretamente a qualidade da resposta obtida. Quanto mais claras forem as instruções, mais adequada tende a ser a resposta.

7.2 Como a qualidade do prompt influenciou os resultados?

Os prompts mais detalhados geraram respostas mais completas, organizadas e adequadas ao objetivo proposto. Já os prompts simples produziram respostas corretas, porém menos específicas.

7.3 Em quais situações a ferramenta foi mais útil?

A ferramenta foi mais útil para explicar conceitos de Computação e gerar exemplos de código que serviram como ponto de partida para a realização das atividades.

7.4 Em quais situações foi necessário verificar ou corrigir respostas?

Foi necessário verificar principalmente os exemplos de código, executando testes para confirmar que o programa funcionava corretamente e que os resultados estavam de acordo com o esperado.

7.5 Você utilizaria essa ferramenta em disciplinas futuras? Por quê?

Sim. A ferramenta pode auxiliar na compreensão de conteúdos, na resolução de dúvidas e na geração de exemplos. No entanto, é importante revisar e validar as respostas antes de utilizá-las.